

PLATAFORMA ADMINISTRATIVA PARA LA PARROQUIA SAN PEDRO

NOLASCO SPRINT 8

Diego Andre Calderón Salazar – 241263

Pedro Julio Caso – 241286

Javier Sebastián Alvarado Monzón – 24546

Hugo Méndez Lee – 241265

José Miguel Rosas Guerra – 241274

Universidad del Valle de Guatemala

Ingeniería en Software 2 - Sección 30

Objetivo del Sprint

El objetivo del Sprint 8 es consolidar la seguridad, mantenibilidad y preparación para producción de la plataforma, al mismo tiempo que se completan funcionalidades prioritarias para ministros y usuarios externos. Durante este sprint se implementarán las notificaciones de periodos de ausencia y de cambios de turno para el coordinador, así como una página pública que permita consultar horarios de misas, eventos e información parroquial sin iniciar sesión. En paralelo, se atenderá la deuda técnica de mayor riesgo mediante controles de acceso por rol, eliminación de secretos de respaldo, validación de variables de entorno, fortalecimiento del flujo de integración continua y ampliación de las pruebas del frontend. También se extenderán las pruebas de seguridad, se realizarán pruebas de volumen e inundación sobre la base de datos y se implementará una mejora de experiencia de usuario validada con usuarios reales. Finalmente, el trabajo se organizará y dará seguimiento en Jira mediante el Sprint Backlog, el calendario, las métricas de burndown y velocidad, la ejecución del presupuesto, la retrospectiva, los registros LOGT y la demostración del incremento. Al finalizar, se espera contar con una versión más segura, verificable, accesible y preparada para el cierre del proyecto.

Historias de usuario identificadas

Sacerdote

HU-01. Como sacerdote, quiero poder utilizar la aplicación administrativa sin problema, dado que no tengo mucha experiencia con dispositivos tecnológicos.

HU-02. Como sacerdote, quiero poder aprobar o rechazar solicitudes de reserva de salones con un solo botón desde la app para evitar que las personas tengan que llamarme constantemente y me quiten tiempo.

HU-03. Como sacerdote, quiero ver un calendario consolidado de quienes están sirviendo en cada misa para saber con qué comunidad o coro contaré antes de iniciar la eucaristía.

HU-27. Como sacerdote, quiero que cuando un evento previamente aprobado sea cancelado, el salón quede liberado automáticamente en el sistema, para mantener la disponibilidad de espacios siempre actualizada sin tener que intervenir manualmente.

HU-28. Como sacerdote o rol administrativo autorizado, quiero ser el responsable de registrar a las personas dentro del sistema, para garantizar que únicamente miembros verificados de la parroquia tengan un perfil activo en la plataforma y evitar accesos de personas externas no autorizadas.

Coordinador de Ministros

HU-04. Como coordinador de ministros, quiero poder acceder rápidamente a la información de los ministros y en tiempo real para ajustar las asignaciones de tareas.

HU-05. Como coordinador de ministros, quiero que los ministros puedan realizar sus responsabilidades sin necesidad de notificarles constantemente.

HU-06. Como coordinador de ministros, quiero poder asignar las tareas correspondientes de forma rápida y eficiente sin necesidad de invertir tanto tiempo en realizarlo.

HU-07. Como coordinador de ministros, me gustaría poder ver los horarios personales de los ministros para poder organizar mejor las tareas parroquiales y no les afecte a ellos en su día a día.

HU-08. Como coordinador de ministros, quisiera reducir la cantidad de errores que cometo al momento de realizar la asignación de tareas de los ministros.

HU-09. Como coordinador de ministros, quiero que el sistema me alerte si estoy asignando a un ministro que está enfermo o que ya ha servido demasiadas veces en el mes para garantizar una rotación justa y humana.

HU-10. Como coordinador de ministros, quiero recibir una notificación inmediata si un voluntario marca que no podrá asistir a su turno de última hora para poder buscar un reemplazo rápidamente.

HU-30 Como coordinador de ministro, quiero recibir una notificación cuando un ministro realice un cambio de turno con otro ministro, por el motivo en el que se encuentre

Coordinador de Grupos

HU-12. Como coordinador de grupos, deseo poder almacenar la información de forma organizada para hacer más eficiente la distribución de trabajo.

HU-14. Como coordinador de grupos, me interesa que sea una aplicación poco saturada y con botones conocidos con la finalidad de agilizar las actividades a realizar.

HU-17. Como coordinador de grupos, quiero ver quién reservó un salón específico para poder contactarlo directamente en caso de que necesite coordinar un intercambio de espacio.

Usuario con Rol de Coordinador

HU-13. Como coordinador, quiero poder apartar salones enviando solicitud al sacerdote sin tener que comunicarme verbalmente con él.

HU-16. Como coordinador, quiero que el sistema bloquee automáticamente los horarios ya reservados en otros salones para que no ocurran duplicaciones donde dos grupos lleguen al mismo lugar al mismo tiempo.

HU-18. Como coordinador, quiero poder apartar un salón de manera rápida y efectiva para eventos específicos.

HU-29. Como coordinador, quiero poder ver en tiempo real la disponibilidad de los salones por fecha y hora, para planificar actividades y eventos sin generar conflictos de espacio ni solapamientos con reservas de otros grupos.

Ministro

HU-19. Como ministro, quiero que la aplicación tenga casillas claras para guardar mi información con el fin de que el proceso de registro sea claro y ordenado.

HU-20. Como ministro, me gustaría tener alguna herramienta digital que me recuerde con anticipación mis servicios en lugar de tener que confiar en mi memoria o calendarios físicos.

HU-21. Como ministro, quisiera tener un lugar donde poder ver todos mis servicios del mes en un mismo lugar y así organizarme mejor.

HU-22. Como ministro, quiero poder confirmar mi asistencia al servicio directamente en la notificación del recordatorio para que mi coordinador sepa que sí recibí la información y que estaré presente.

HU-23. Como ministro, quiero tener la opción de solicitar un cambio de turno con otro compañero dentro de la plataforma para que el sistema actualice al nuevo titular automáticamente sin que el coordinador deba hacerlo a mano.

HU-31 Como ministro, quiero mandra una notificación al coordinador, de periodos de ausencia.

Usuario General (todos los roles)

HU-24. Como usuario del sistema, quiero que mi sesión se mantenga activa durante un período razonable sin necesidad de volver a autenticarme, para no interrumpir mi flujo de trabajo constantemente.

HU-25. Como usuario, quiero ver una página informativa cuando acceda a una ruta inexistente (404), para entender que me he equivocado de URL y poder regresar fácilmente.

HU-26. Como usuario, quiero recibir una notificación clara cuando mi sesión haya expirado, para saber que debo iniciar sesión nuevamente sin confundirme con un error del sistema.

HU-32 Como usuario, quiero sin la necesidad de iniciar sesión, poder ver los horarios de misa, evento y quienes son en la parroquia.

Product Backlog

Pila del Producto

ID	Rol	Historia	Sprint
HU-00	—	Documentación/Gestión (no funcional): contenedor de tareas de informe, métricas, retroalimentación y tablas del sprint.	Recurrente (Sprints 1-8)
HU-01	Sacerdote	Como sacerdote, quiero poder utilizar la aplicación administrativa sin problema, dado que no tengo mucha experiencia con dispositivos tecnológicos.	Sprint 3
HU-02	Sacerdote	Como sacerdote, quiero poder aprobar o rechazar solicitudes de reserva de salones con un solo botón desde la app para evitar que las personas tengan que llamarme constantemente y me quiten tiempo.	Sprint 1-2 (refinado Sprint 3)
HU-03	Sacerdote	Como sacerdote, quiero ver un calendario consolidado de quienes están sirviendo en cada misa para saber con qué comunidad o coro contaré antes de iniciar la eucaristía.	Sprint 4
HU-04	Coord. Ministros	Como coordinador de ministros, quiero poder acceder rápidamente a la información de los ministros y en tiempo real para ajustar las asignaciones de tareas.	Sprint 2
HU-05	Coord. Ministros	Como coordinador de ministros, quiero que los ministros puedan realizar sus responsabilidades sin necesidad de notificarles constantemente.	Sprint 4
HU-06	Coord. Ministros	Como coordinador de ministros, quiero poder asignar las tareas correspondientes de forma rápida y eficiente sin necesidad de invertir tanto tiempo en realizarlo.	Sprint 2
HU-07	Coord. Ministros	Como coordinador de ministros, me gustaría poder ver los horarios personales de los ministros para poder organizar mejor las tareas parroquiales y no les afecte a ellos en su día a día.	Sprint 4
HU-08	Coord. Ministros	Como coordinador de ministros, quisiera reducir la cantidad de errores que cometo al momento de realizar la asignación de tareas de los ministros.	Sprint 2-3 (validación Sprint 5, PR #37)
HU-09	Coord. Ministros	Como coordinador de ministros, quiero que el sistema me alerte si estoy asignando a un ministro que está enfermo o que ya ha servido demasiadas veces en el mes para garantizar una rotación justa y humana.	Sprint 7 (P1/P2)
HU-10	Coord. Ministros	Como coordinador de ministros, quiero recibir una notificación inmediata si un voluntario marca que no podrá asistir a su turno de última	Sprint 6 (M1/P2)

		hora para poder buscar un reemplazo rápidamente.	
HU-12	Coord. Grupos	Como coordinador de grupos, deseo poder almacenar la información de forma organizada para hacer más eficiente la distribución de trabajo.	Sprint 2
HU-13	Coord. Grupos	Como coordinador de grupos, quiero poder apartar salones enviando solicitud al sacerdote sin tener que comunicarme verbalmente con él.	Sprint 1-2 (evento asociado Sprint 3)
HU-14	Coord. Grupos	Como coordinador de grupos, me interesa que sea una aplicación poco saturada y con botones conocidos con la finalidad de agilizar las actividades a realizar.	Sprint 3
HU-15	Coord. Grupos	Como coordinador de grupos, quiero realizar un check-in de un salón mediante un código QR para que en el sistema salga automáticamente que el salón ya está en uso.	—
HU-16	Coord. Grupos	Como coordinador de grupos, quiero que el sistema bloquee automáticamente los horarios ya reservados en otros salones para que no ocurran duplicaciones donde dos grupos lleguen al mismo lugar al mismo tiempo.	Sprint 3
HU-17	Coord. Grupos	Como coordinador de grupos, quiero ver quién reservó un salón específico para poder contactarlo directamente en caso de que necesite coordinar un intercambio de espacio.	Sprint 3
HU-18	Coord. Ministros	Como coordinador de ministros, quiero poder apartar un salón de manera rápida y efectiva para eventos específicos.	Sprint 3
HU-19	Ministro	Como ministro, quiero que la aplicación tenga casillas claras para guardar mi información con el fin de que el proceso de registro sea claro y ordenado.	Sprint 1-2
HU-20	Ministro	Como ministro, me gustaría tener alguna herramienta digital que me recuerde con anticipación mis servicios en lugar de tener que confiar en mi memoria o calendarios físicos.	Sprint 4
HU-21	Ministro	Como ministro, quisiera tener un lugar donde poder ver todos mis servicios del mes en un mismo lugar y así organizarme mejor.	Sprint 2 (mis-reservas) / Sprint 4 (calendario)
HU-22	Ministro	Como ministro, quiero poder confirmar mi asistencia al servicio directamente en la notificación del recordatorio para que mi coordinador sepa que sí recibí la información y que estaré presente.	Sprint 5 (PR #36)
HU-23	Ministro	Como ministro, quiero tener la opción de solicitar un cambio de turno con otro compañero dentro de la plataforma para que el sistema actualice al nuevo titular automáticamente sin que el coordinador deba hacerlo a mano.	Sprint 7 (D3/D5)
HU-24	Usuario general	Como usuario del sistema, quiero que mi sesión se mantenga activa durante un período	Sprint 3

		razonable sin necesidad de volver a autenticarme, para no interrumpir mi flujo de trabajo constantemente.	
HU-25	Usuario general	Como usuario, quiero ver una página informativa cuando acceda a una ruta inexistente (404), para entender que me he equivocado de URL y poder regresar fácilmente.	Sprint 3
HU-26	Usuario general	Como usuario, quiero recibir una notificación clara cuando mi sesión haya expirado, para saber que debo iniciar sesión nuevamente sin confundirme con un error del sistema.	Sprint 3
HU-27	Sacerdote	Como sacerdote, quiero que cuando un evento previamente aprobado sea cancelado, el salón quede liberado automáticamente en el sistema, para mantener la disponibilidad de espacios siempre actualizada sin tener que intervenir manualmente.	Sprint 3
HU-28	Sacerdote	Como sacerdote [o rol administrativo autorizado], quiero ser el responsable de registrar a las personas dentro del sistema, para garantizar que únicamente miembros verificados de la parroquia tengan un perfil activo en la plataforma y evitar accesos de personas externas no autorizadas.	Sprint 3
HU-29	Coordinador (genérico)	Como coordinador, quiero poder ver en tiempo real la disponibilidad de los salones por fecha y hora, para planificar actividades y eventos sin generar conflictos de espacio ni solapamientos con reservas de otros grupos.	Sprint 3 (backend) / Sprint 4 (badge dinámico)
HU-30	Coordinador de Ministros	Como coordinador de ministros, quiero recibir una notificación cuando un ministro realice un cambio de turno con otro ministro, para mantener trazabilidad de las reasignaciones.	Sprint 8 (backend #10 / frontend #18)
HU-31	Ministro	Como ministro, quiero notificar al coordinador mis periodos de ausencia, para evitar asignaciones durante las fechas en que no puedo servir.	Sprint 8 (backend #9 / frontend #17)
HU-32	Usuario general	Como usuario, quiero consultar sin iniciar sesión los horarios de misas, eventos y la información de la parroquia.	Sprint 8 (backend #13 / frontend #19)
—	Solicitante	Como solicitante, quiero poder cancelar mi propia reserva de salón sin depender de la aprobación del Admin o del Sacerdote, para tener control directo sobre mis solicitudes cuando ya no las necesito.	Sprint 6 (D3)

Lista de Tareas del Sistema con prioridad

Historia/Épica	Tipo	Qué incluye	SP	Prioridad
----------------	------	-------------	----	-----------

HU-30: Notificación de cambio de turno	HU funcional	Backend y frontend para notificar al coordinador cuando un ministro complete un cambio de turno.	6	Alta
HU-31: Periodos de ausencia	HU funcional	Backend y frontend para que el ministro comunique al coordinador sus periodos de ausencia.	6	Alta
HU-32: Portal público	HU funcional	Backend y frontend de una página pública con horarios de misas, eventos e información parroquial.	12	Alta
Deuda técnica - Seguridad y RBAC	No funcional (refactorización)	DT-05 a DT-08: roles en rutas, secretos JWT y validación de variables de entorno.	9	Crítica
Deuda técnica - CI y pruebas	No funcional (refactorización)	DT-04 y DT-12: build, lint, pruebas y ampliación de cobertura del frontend.	8	Alta
Refactor - Build y endurecimiento HTTP	Corrección y seguridad	Corregir el build; añadir rate limiting, cabeceras seguras y ajuste de CORS.	5	Alta
Pruebas de seguridad	No funcional (pruebas)	Extender las pruebas de seguridad pendientes del plan maestro y documentar mitigaciones.	5	Alta
Pruebas de volumen e inundación	No funcional (pruebas)	Evaluar el comportamiento de la base de datos bajo volumen e inundación.	6	Alta
Pruebas y mejora de UX	No funcional (UX)	Implementar una mejora derivada del Sprint 7 y validarla con usuarios reales.	3	Media
Investigación de imágenes	Investigación	Evaluar un servicio externo para alojar y servir imágenes reales de los espacios.	2	Media
Comparación de RNF	No funcional (informe)	Comparar las pruebas no funcionales con los requisitos definidos durante el diseño.	2	Media
HU-00: Gestión y documentación	No funcional (contenedor)	Jira, Sprint Backlog, calendario, métricas, presupuesto, incremento, retrospectiva, demo y LOGT.	17	Alta

Sprint Backlog

Tareas a realizar

#	Tarea	Épica	Descripción	S P	Responsable	Fecha límite
---	-------	-------	-------------	--------	-------------	--------------

1	Gestión del sprint en Jira	HU-00: Gestión	Elaborar la pila y el calendario del Sprint 8; cargar las tareas en Jira antes de iniciar y mantener el flujo real de trabajo en el tablero.	1	Diego Calderón	18/09/2026
2	Corregir build de backend y frontend	Corrección de defecto	Corregir el error TS2345 de reserva.controller.ts y los errores de configuración y tipado de UIComponents.test.tsx para recuperar el build.	2	Pedro Caso	18/09/2026
3	RBAC en rutas de grupos	Deuda técnica (DT-05)	Aplicar requireRole a POST, PUT y DELETE de grupos y añadir pruebas negativas por rol.	3	Diego Calderón	19/09/2026
4	RBAC en rutas de tareas	Deuda técnica (DT-06)	Centralizar la autorización de las mutaciones de tareas, restringirlas por rol o propiedad y probar cada rol.	3	Diego Calderón	19/09/2026
5	CI: build, lint y pruebas	Deuda técnica (DT-04)	Agregar al CI los jobs de build, lint y pruebas del frontend y exigir el check antes de fusionar en develop.	2	Pedro Caso	19/09/2026
6	Secretos JWT y variables de entorno	Deuda técnica (DT-07 y DT-08)	Eliminar secretos de respaldo, centralizar la configuración, validar las variables obligatorias y detener el arranque si faltan.	3	Diego Calderón	20/09/2026
7	Rate limiting, cabeceras y CORS	Deuda técnica (refactorización)	Incorporar rate limiting al login, cabeceras HTTP seguras y una política CORS válida para cada entorno.	3	Pedro Caso	20/09/2026
8	Servicio externo para imágenes	Investigación	Investigar alternativas para alojar y servir fotografías reales de los espacios cuando el cliente las entregue; documentar la recomendación.	2	Hugo Méndez	20/09/2026
9	Backend: periodos de ausencia	HU-31: Periodos de ausencia	Implementar el flujo de backend para que un ministro registre un periodo de ausencia y	3	Miguel Rosas	20/09/2026

			se notifique al coordinador.			
10	Backend: notificación de cambio de turno	HU-30: Notificación de cambio de turno	Extender responderCambioTurno para generar la notificación correspondiente al coordinador.	3	Pedro Caso	21/09/2026
11	Cobertura de pruebas del frontend	Deuda técnica (DT-12)	Ampliar la cobertura del frontend a páginas, formularios y hooks prioritarios; coordinar su ejecución automática con DT-04.	6	Hugo Méndez	21/09/2026
12	Extensión de pruebas de seguridad	Pruebas de seguridad	Ejecutar los casos de seguridad pendientes del plan maestro, revisar exposición de MariaDB y configuración CORS, y documentar hallazgos y mitigaciones.	5	Miguel Rosas	21/09/2026
13	Backend: portal público	HU-32: Portal público	Implementar los servicios requeridos por el portal público para consultar horarios de misas, eventos e información parroquial sin autenticación.	5	Diego Calderón	22/09/2026
14	Plan de refactorización	HU-00: Documentación	Planificar la refactorización del Sprint 8 y los siguientes; incluir DT-02, DT-03, DT-09, DT-11 y AT-05 como pendientes reprogramados.	1	Diego Calderón	23/09/2026
15	Mejora y validación de UX	Pruebas de UX	Elegir una fricción detectada en Sprint 7, implementar la mejora y validarla con usuarios reales mediante una nueva sesión.	3	Javier Alvarado	23/09/2026
16	Pruebas de volumen e inundación	Pruebas de volumen/inundación	Ejecutar pruebas de volumen e inundación sobre la base de datos con una herramienta apropiada; documentar resultados y acciones de mejora.	6	Hugo Méndez	23/09/2026

17	Frontend: periodos de ausencia	HU-31: Periodos de ausencia	Construir la interfaz que consume el backend de HU-31 y permite al ministro comunicar su ausencia al coordinador.	3	Miguel Rosas	23/09/2026
18	Frontend: notificación de cambio de turno	HU-30: Notificación de cambio de turno	Integrar en el frontend el flujo de notificación al coordinador cuando se complete un cambio de turno.	3	Pedro Caso	24/09/2026
19	Frontend: portal público	HU-32: Portal público	Construir la página pública que consume el backend de HU-32 y presenta horarios e información parroquial sin iniciar sesión.	7	Javier Alvarado	24/09/2026
20	Comparación de RNF	HU-00: Informe	Comparar los resultados acumulados de pruebas no funcionales con los requisitos definidos en Ingeniería de Software 1 y documentar brechas.	2	Miguel Rosas	26/09/2026
21	Revisión de rúbrica	HU-00: Gestión	Revisar el informe punto por punto contra la sección Evaluación de la guía del Sprint 8.	1	Pedro Caso	27/09/2026
22	Burndown y velocidad	HU-00: Informe	Capturar e interpretar el burndown de Jira y agregar el Sprint 8 a la gráfica histórica de velocidad.	1	Javier Alvarado	27/09/2026
23	Presupuesto y tiempo	HU-00: Informe	Actualizar el costo y tiempo consumidos, en valores absolutos y porcentajes, usando el cierre del Sprint 7 como base.	1	Javier Alvarado	27/09/2026
24	Prueba integral del sistema	HU-00: QA	Probar el sistema de punta a punta con los cinco roles antes de la demostración e informar cualquier defecto.	1	Hugo Méndez	27/09/2026
25	Product Backlog, incremento y resultados	HU-00: Informe	Actualizar el Product Backlog y documentar el incremento, las tareas concluidas, en proceso y no concluidas, verificándolas contra Jira y el repositorio.	3	Miguel Rosas	27/09/2026

26	Retrospectiva del sprint	HU-00: Documentación	Reflexionar sobre el desempeño del equipo, los aspectos positivos y negativos y las acciones para mejorar en el siguiente sprint.	1	Diego Calderón	28/09/2026
27	LOGT - Diego Calderón	HU-00: Documentación	Completar el formulario LOGT PSP0 individual con todas las sesiones y actividades del sprint.	1	Diego Calderón	28/09/2026
28	LOGT - Pedro Caso	HU-00: Documentación	Completar el formulario LOGT PSP0 individual con todas las sesiones y actividades del sprint.	1	Pedro Caso	28/09/2026
29	LOGT - Javier Alvarado	HU-00: Documentación	Completar el formulario LOGT PSP0 individual con todas las sesiones y actividades del sprint.	1	Javier Alvarado	28/09/2026
30	Demo a usuarios finales	HU-00: Gestión	Presentar el incremento a usuarios finales reales, recopilar sus comentarios y documentar la evidencia de socialización.	2	Javier Alvarado	28/09/2026
31	LOGT - Hugo Méndez	HU-00: Documentación	Completar el formulario LOGT PSP0 individual con todas las sesiones y actividades del sprint.	1	Hugo Méndez	28/09/2026
32	LOGT - Miguel Rosas	HU-00: Documentación	Completar el formulario LOGT PSP0 individual con todas las sesiones y actividades del sprint.	1	Miguel Rosas	28/09/2026

Total de SP planificados: 81

Calendario del Sprint

Fecha	Tareas planificadas	Responsable(s)	SP del día	Horas del día
18/09/2026	Sprint Backlog + Calendario + Organización en Jira (#1) - Corregir build roto de backend y frontend (#2)	Diego, Pedro	3	6h
19/09/2026	RBAC en rutas de grupos (#3) - RBAC en rutas de tareas (#4) - CI con build, lint y pruebas de frontend (#5)	Diego, Pedro	8	16h
20/09/2026	Secretos JWT y variables de entorno (#6) - Rate limiting, cabeceras y CORS (#7) - Investigación de imágenes (#8) - HU-31 Backend (#9)	Diego, Pedro, Hugo, Miguel	11	22h
21/09/2026	HU-30 Backend (#10) - Cobertura de pruebas de frontend (#11) - Seguridad: extensión (#12)	Pedro, Hugo, Miguel	14	28h
22/09/2026	Landing page Backend (#13)	Diego	5	10h
23/09/2026	Plan de refactorización (#14) - Mejora y validación UX (#15) - Volumen/inundación (#16) - HU-31 Frontend (#17)	Diego, Javier, Hugo, Miguel	13	26h
24/09/2026	HU-30 Frontend (#18) - Landing page Frontend (#19)	Pedro, Javier	10	20h
26/09/2026	Comparación de RNF y pruebas no funcionales (#20)	Miguel	2	4h
27/09/2026	Revisión de rúbrica (#21) - Burndown y velocidad (#22) - Presupuesto (#23) - QA integral (#24) - Product Backlog, incremento y resultados (#25)	Pedro, Javier, Hugo, Miguel	7	14h
28/09/2026	Retrospectiva (#26) - Demo a usuarios finales (#30) - LOGT del equipo (#27, #28, #29, #31 y #32)	Diego, Javier y todo el equipo	8	16h
29/09/2026	Cierre del sprint y demostración final	Todo el equipo	-	-

Total de puntos planificados: 81 / Total de horas planificadas: 162h

Incremento: Resultados del Sprint

Link del Repositorio:

https://github.com/hmndzzl/Software_Proyecto.git

Tareas Concluidas

Clave	Resumen	Asignado a	Story Points	Estado
D1	Gestión del sprint en Jira	Diego Calderón	1	Finalizada
D2	RBAC en rutas de grupos	Diego Calderón	3	Finalizada
D3	RBAC en rutas de tareas	Diego Calderón	3	Finalizada
D4	Secretos JWT y variables de entorno	Diego Calderón	3	Finalizada
D5	Backend HU-32 — Portal público	Diego Calderón	5	Finalizada
D6	Plan de refactorización	Diego Calderón	1	Finalizada
D7	Retrospectiva del sprint	Diego Calderón	1	Finalizada
D8	LOGT propio	Diego Calderón	1	Finalizada
P1	Corregir build de backend y frontend	Pedro Caso	2	Finalizada
P2	CI — Build, lint y pruebas	Pedro Caso	2	Finalizada
P3	Rate limiting, headers y CORS	Pedro Caso	3	Finalizada
P4	Backend HU-30 — Notificación de cambio de turno	Pedro Caso	3	Finalizada
P5	Frontend HU-30 — Notificación de cambio de turno	Pedro Caso	3	Finalizada
P6	Revisión de rúbrica	Pedro Caso	1	Finalizada
P7	LOGT propio	Pedro Caso	1	Finalizada
M1	Backend HU-31 — Periodos de ausencia	Miguel Rosas	3	Finalizada

M2	Extensión de pruebas de seguridad	Miguel Rosas	5	Finalizada
M3	Frontend HU-31 — Periodos de ausencia	Miguel Rosas	3	Finalizada
M4	Comparación de RNF	Miguel Rosas	2	Finalizada
M5	Product Backlog, incremento y resultados	Miguel Rosas	3	Finalizada
M6	LOGT propio	Miguel Rosas	1	Finalizada
H1	Investigación — Servicio externo para imágenes	Hugo Méndez	2	Finalizada
H2	Cobertura de pruebas del frontend	Hugo Méndez	6	Finalizada
H3	Pruebas de volumen e inundación	Hugo Méndez	6	Finalizada
H4	QA — Prueba integral del sistema	Hugo Méndez	1	Finalizada
H5	LOGT propio	Hugo Méndez	1	Finalizada
J1	Mejora y validación de UX	Javier Alvarado	3	Finalizada
J2	Frontend HU-32 — Portal público	Javier Alvarado	7	Finalizada
J3	Métricas — Burndown y velocidad	Javier Alvarado	1	Finalizada
J4	Presupuesto y tiempo	Javier Alvarado	1	Finalizada
J5	LOGT propio	Javier Alvarado	1	Finalizada
J6	Demo a usuarios finales	Javier Alvarado	2	Finalizada
Total de puntos realizados		81		

Tareas en Proceso

Ninguna tarea quedó en proceso al cierre del sprint. El equipo completó el 100% de las tareas planificadas.

Tareas no Concluidas

Ninguna tarea fue descartada ni quedó sin concluir. Se completaron todas las tareas del sprint.

Pruebas de seguridad

Se evaluó el sistema publicado para comprobar la protección de sesiones, la presencia de cuentas de prueba, la exposición de MariaDB y la respuesta de la API ante errores. Las pruebas manuales y automatizadas identificaron incumplimientos en PS-SEG-09, PS-SEG-10, PS-SEG-11 y la parte A de PS-SEG-12. Los errores 401 y 404 probados no mostraron información interna. La carga moderada se completó sin errores.

Alcance y método

El objetivo fue <https://parroquiasanpedronolasco.org.gt>, con API bajo /api/api. La revisión manual fue documentada por Miguel Rosas el 23 de septiembre de 2026, con ZAP 2.17.0. Los headers fechados el 24 de septiembre en GMT corresponden a la noche del 23 en Guatemala. La ejecución automatizada de seguridad y carga se realizó el 25 de septiembre; PS-SEG-11 se completó el 27 de septiembre.

ZAP se utilizó para inspeccionar solicitudes y respuestas y realizar análisis pasivo. La automatización empleó Python y ZAP en Docker; la prueba de red usó conexiones TCP directas con Python, ya que MariaDB no utiliza el protocolo HTTP. La carga se generó con Grafana k6 2.3.0 en Docker. No se aplicaron cambios al código ni al servidor durante estas verificaciones.

Los casos proceden del Plan Maestro de Pruebas, páginas 10 y 11. PS-SEG-01 a PS-SEG-08 pertenecen al trabajo previo de Sprint7 y no se reejecutaron íntegramente. El documento manual identifica la versión a531f04; la revisión del código utiliza origin/main en a531f04d9d7c637f68d23bc64d08c7afe58ec9b8. No se verificó independientemente el commit instalado en el servidor.

Resultados de seguridad

Caso	Resultado	Evidencia principal
PS-SEG-09	No cumple	Cookie de refresh sin Secure.
PS-SEG-10	No cumple	Acceso con credenciales conocidas de prueba.
PS-SEG-11	No cumple	Tres conexiones externas a MariaDB en TCP 3306.
PS-SEG-12 A	No cumple	Headers de seguridad ausentes o inconsistentes.
PS-SEG-12 B	Cumple en lo probado	Errores 401 y 404 sin detalles internos; 500 no evaluado.

La división A y B facilita el análisis de PS-SEG-12. El resultado global de este caso es no cumple por las deficiencias de la parte A; la respuesta adecuada de dos errores no compensa esas ausencias.

PS SEG 09 Protección de la cookie de refresh

Se inspeccionó el header Set-Cookie de una respuesta exitosa a POST /api/api/auth/login. La revisión manual y la automatizada coincidieron en los siguientes atributos:

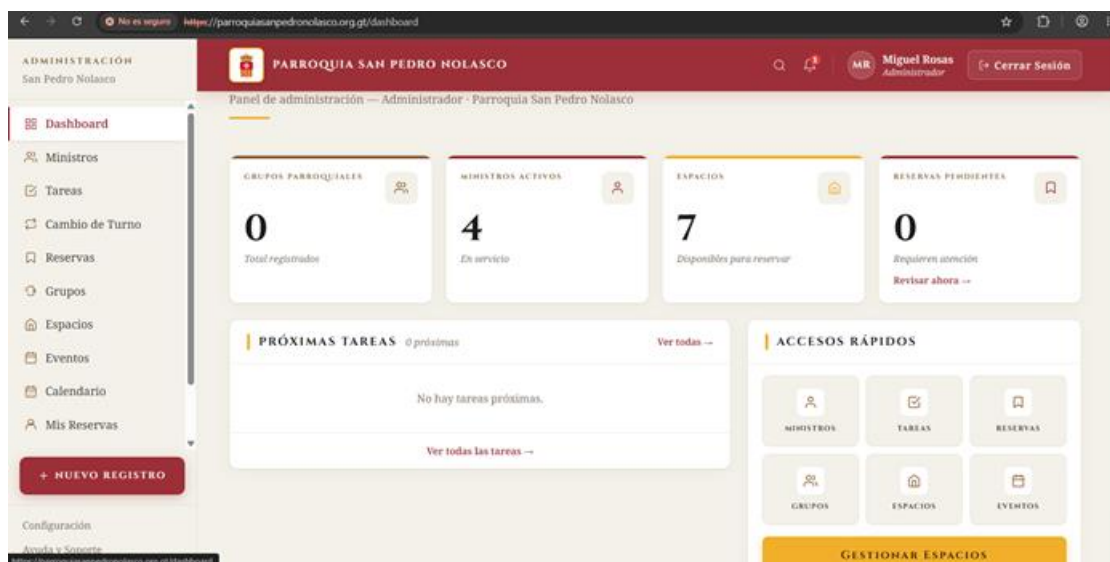
Atributo exigido	Observado	Evaluación
HttpOnly	Presente	Cumple
Secure	Ausente	No cumple
SameSite=Strict	Presente	Cumple

Conclusión. La cookie no cumple todos los atributos exigidos. Aunque el login se realizó mediante HTTPS, la cookie no declara que deba enviarse exclusivamente mediante conexiones seguras. No se comprobó una interceptación ni se intentó transmitirla por HTTP.

Solución propuesta. Configurar Secure junto con HttpOnly y SameSite=Strict en producción, verificar el comportamiento detrás del proxy y repetir el login para comprobar el nuevo Set-Cookie. Los tokens de las capturas originales no se reproducen en este informe.

PS SEG 10 Cuentas de prueba en producción

La prueba manual registró autenticación exitosa y acceso al dashboard con una cuenta identificada como de prueba. La ejecución automática confirmó que ministro@parroquia.com aceptó su contraseña conocida y permitió consultar GET /api/api/notificaciones con respuesta 200. Se trata de evidencias de cuentas distintas, no de una única sesión compartida entre ambas pruebas.



Conclusión. Existe al menos una cuenta de prueba activa con contraseña conocida en el entorno publicado. Esto basta para declarar el incumplimiento; no se revisó el inventario completo de usuarios. El indicador del navegador en la captura no se utiliza como diagnóstico del certificado del servidor.

Solución propuesta. Retirar o deshabilitar las cuentas de desarrollo, cambiar las credenciales que deban conservarse, invalidar sus sesiones y evitar la carga de usuarios de prueba en producción. Verificar después que las credenciales anteriores ya no permiten acceder.

PS SEG 11 Exposición de MariaDB

El 27 de septiembre se verificó la IP 137.184.193.76, proporcionada como servidor del sistema. Se realizaron tres conexiones desde el equipo externo de pruebas al puerto TCP 3306 y se leyó únicamente el saludo inicial del servicio. No se enviaron credenciales ni consultas SQL.

Intento	Hora UTC	Conexión	Duración
1	20:51:47	Aceptada	187 ms
2	20:51:48	Aceptada	172 ms
3	20:51:49	Aceptada	156 ms

En los tres intentos se recibieron 94 bytes y un saludo compatible con MySQL/MariaDB. El servicio anunció la versión 11.8.8-MariaDB-ubu2404. La evidencia se conserva en resultados.json de la ejecución ps-seg-11-20260927T205150Z.

Conclusión. PS-SEG-11 no cumple: MariaDB aceptó conexiones desde el origen externo utilizado. La prueba confirma exposición de red desde ese origen; no demuestra acceso a los datos, autenticación sin contraseña ni accesibilidad desde cualquier dirección de Internet. La IP pública de salida del equipo no quedó registrada y no se revisaron las reglas efectivas del firewall.

Solución propuesta. Retirar la publicación pública de 3306 cuando no sea necesaria, mantener la base de datos en una red interna accesible por el backend y restringir cualquier acceso administrativo a orígenes autorizados. Tras el ajuste, repetir la prueba externa y comprobar que la aplicación conserva su conexión interna.

Hallazgos adicionales de configuración

Puertos de MariaDB y Adminer

El archivo versionado `app/docker-compose.example.yml` publica MariaDB como 3306:3306 y Adminer como `${ADMINER_PORT}:8080`. El archivo `app/docker-compose.yml` no está versionado en la referencia revisada; el `compose` local publica MariaDB, pero no permite confirmar la configuración efectiva del servidor. La conexión externa sí confirma el alcance de red observado para MariaDB.

No se comprobó Adminer porque no se conoce su puerto externo. Su publicación en el ejemplo es un riesgo de configuración pendiente de verificación, no evidencia de que esté accesible en producción.

Origen localhost permitido en CORS

`app/backend/src/app.ts` incluye <http://localhost:5173> sin distinguir el entorno. La solicitud automatizada con ese Origin obtuvo Access-Control-Allow-Origin con el mismo valor, aunque respondió 401 por falta de autenticación. Se propone separar los orígenes permitidos por entorno. Permitir un origen en CORS no equivale a permitir acceso sin credenciales.

PS SEG 12 Headers y respuestas de error

La inspección manual y la automatizada mostraron el mismo patrón. Para esta evaluación se comprobaron X-Content-Type-Options: nosniff y Strict-Transport-Security; también se registró Content-Security-Policy. Esta selección concreta es un criterio operativo de la prueba, ya que el caso original no enumera todos los headers.

Respuesta	nosniff	HSTS	CSP
Notificaciones 200	Ausente	Ausente	Ausente
Login inválido 401	Ausente	Ausente	Ausente
Ruta inexistente 404	Presente	Ausente	default-src 'none'

Conclusión. La parte A no cumple. Además, Server anuncia nginx/1.18.0 (Ubuntu) y X-Powered-By identifica Express. Esta divulgación facilita identificar tecnologías, pero por sí sola no demuestra una vulnerabilidad explotable. Se propone aplicar headers consistentes en API y proxy, definir HSTS según el despliegue HTTPS y reducir la información de versiones.

```
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Thu, 24 Sep 2026 05:51:10 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 713
Connection: keep-alive
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
ETag: W/"2c9-YUKTjowhm+9VxGSuxYLLmkuYM2k"
```

```
[{"id":2,"mensaje":"Reunión de coordinadores este lunes 25 de mayo a las 9am en la Sala de Reuniones.,"fecha":"2026-05-21","tipo":"global",
"remite_id":6,"grupo_id":null,"evento_id":null,"requiere_confirmacion":0,"leida":0,"asistencia_confirmada":0,"inasistencia_reportada":0,
"motivo_excusa":null,"remite_nombre":"Padre Test","evento_descripcion":null}, {"id":1,"mensaje":
"Bienvenidos al nuevo sistema de gestión parroquial. ¡Ya pueden usarlo!","fecha":"2026-05-10","tipo":"global","remite_id":1,"grupo_id":
null,"evento_id":null,"requiere_confirmacion":0,"leida":1,"asistencia_confirmada":0,"inasistencia_reportada":0,"motivo_excusa":null,
"remite_nombre":"Diego Calderon","evento_descripcion":null}]
```

PS SEG 12 Manejo de errores

El login incorrecto respondió 401 con “Credenciales inválidas”. La ruta inexistente respondió 404 con “Cannot GET” y la ruta solicitada. No se observaron detalles internos de ejecución, consultas SQL ni rutas de archivos del servidor. Una ruta HTTP solicitada no equivale a una ruta interna del sistema de archivos.

```

HTTP/1.1 401 Unauthorized
Server: nginx/1.18.0 (Ubuntu)
Date: Thu, 24 Sep 2026 06:00:16 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 37
Connection: keep-alive
X-Powered-By: Express
Access-Control-Allow-Origin: https://parroquiasanpedronolasco.org.gt
Vary: Origin
Access-Control-Allow-Credentials: true
ETag: W/"25-A1tikMeeIJNgVMSyKmJJFS1+RX0"

{"mensaje": "Credenciales inválidas"}

```

```

HTTP/1.1 404 Not Found
Server: nginx/1.18.0 (Ubuntu)
Date: Thu, 24 Sep 2026 06:06:21 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 166
Connection: keep-alive
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Content-Security-Policy: default-src 'none'
X-Content-Type-Options: nosniff

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Error</title>
</head>
<body>
<pre>Cannot GET /api/ruta-inexistente-prueba</pre>
</body>
</html>

```

Conclusión. La parte B cumple en los errores 401 y 404 comprobados. No se provocaron errores 500; su manejo deberá evaluarse en un entorno de pruebas controlado.

Prueba de carga controlada

Para atender el componente de volumen de la guía, se ejecutó una carga moderada sobre GET /api/api/notificaciones el 25 de septiembre. k6 realizó un login y compartió esa sesión entre los usuarios virtuales. Se configuró una rampa de 60 segundos, con etapas de 20 segundos hacia 1, 2 y 3 usuarios, y pausas de un segundo por usuario.

El máximo de usuarios registrado por la métrica muestreada fue 2; 3 era el objetivo final de la rampa y no una carga sostenida. Se establecieron paradas preventivas ante respuestas 429, errores de servidor o de conexión. Los criterios operativos fueron p95 menor de 2 segundos, tasa de errores menor de 5 % y consultas con respuesta 200; estos valores no constituyen un SLA aprobado del proyecto.

Indicador	Resultado
Solicitudes totales	75, incluido el login
Consultas autenticadas	74 respuestas HTTP 200
Errores de consulta	0 %
Tiempo promedio de consulta	94,46 ms
Percentil 95 de consulta	102,23 ms
Tiempo máximo de consulta	136,83 ms

Conclusión. La API respondió correctamente bajo la carga aplicada y cumplió los umbrales de esta ejecución. El resultado no demuestra disponibilidad continua, capacidad máxima ni resistencia a inundación. No se midieron CPU, memoria, conexiones de base de datos ni comportamiento con múltiples cuentas independientes.

Plan de corrección y verificación

Las siguientes acciones son propuestas de seguimiento. No se aplicaron durante las pruebas ni se asignan responsables o fechas que no hayan sido acordados por el equipo.

Hallazgo	Acción propuesta	Criterio para cerrar
MariaDB accesible externamente	Restringir red y puertos publicados.	Falla el acceso externo no autorizado y funciona el acceso interno del backend.
Credenciales de prueba activas	Retirar cuentas o rotar credenciales e invalidar sesiones.	Las credenciales conocidas dejan de funcionar; inventario revisado.
Cookie sin Secure	Completar los flags de la cookie en producción.	Login HTTPS emite HttpOnly, Secure y SameSite=Strict.
Headers inconsistentes	Configurar API y proxy y reducir divulgación de versiones.	Verificación satisfactoria en respuestas 200, 401 y 404.
CORS y Adminer	Separar orígenes por entorno y verificar el acceso administrativo.	localhost excluido de producción y Adminer restringido o no publicado.

Conclusión de la evaluación

La evidencia manual y la automatizada coinciden en los problemas de la cookie, las cuentas de prueba y los headers. La ejecución complementaria confirmó que MariaDB acepta

conexiones externas. Se recomienda atender primero la exposición de la base de datos y las credenciales conocidas, y después repetir los casos afectados para demostrar la corrección.

Los mensajes de error evaluados no expusieron información interna y la carga moderada terminó sin errores. Estos resultados favorables se limitan a los escenarios ejecutados. La evaluación no permite declarar al sistema completamente seguro ni afirmar cumplimiento total de disponibilidad o resistencia a inundación.

Comparación de requisitos no funcionales y pruebas

Usabilidad e interfaz

Requisito no funcional	Pruebas o trabajo relacionado	Resultado actualizado	Verificación pendiente
Sistema simple, legible y con una interfaz limpia y ordenada.	Pruebas unitarias de renderizado, estados vacíos y errores; pruebas UX del plan.	Se documentan comprobaciones automatizadas de componentes. Estas aportan evidencia de funcionamiento, pero no demuestran por sí solas facilidad de uso.	Incorporar resultados de sesiones con usuarios sobre legibilidad, comprensión y organización.
Sistema atractivo, con colores e iconos adecuados.	UX-03, prueba A/B; UX-09, cuestionario UEQ.	Los casos están definidos, pero no se presentan puntuaciones ni resultados específicos en las evidencias revisadas.	Registrar preferencias y criterios de aceptación visual.
Asignar tareas en menos de cinco pasos.	Flujo funcional de asignación y evaluación heurística de tareas.	No se documenta un conteo de pasos ni un tiempo observado para este flujo.	Medir pasos y tiempo al asignar una tarea.
Consistencia al realizar reservas.	Pruebas de reservas, formularios y componentes del frontend.	Se documentan validaciones automatizadas y pruebas del flujo de reservas. La uniformidad del botón y su comportamiento entre vistas no queda demostrada.	Comparar las distintas vistas de reserva.
Facilidad de uso para personas sin experiencia y adultos mayores.	Actividades UX y pruebas automatizadas de interacción mediante teclado.	Se verificaron interacciones de teclado en componentes. Falta evidencia de resultados con los perfiles de usuarios requeridos.	Incorporar participantes, tareas completadas, ayudas y dificultades observadas.
Mostrar mensajes claros ante errores.	Pruebas de estados de error, fallos de red y PS-SEG-12 B.	Se documenta manejo de errores en el frontend. Las respuestas 401 y 404 probadas no revelaron información interna.	Validar comprensión con usuarios y comprobar errores 500 en un entorno controlado.

Rendimiento y concurrencia

Requisito no funcional	Pruebas realizadas	Resultado actualizado	Verificación o acción pendiente
Respuesta rápida y menos de 2 segundos en el 95 % de peticiones normales.	Carga HTTP con k6 y consultas SQL con Sysbench.	La carga moderada de notificaciones obtuvo p95 de 102,23 ms. Sysbench reportó p95 de 4,57 ms, 4,36 s y 8,64 s en escalas pequeña, mediana y grande. Hay degradación importante al aumentar los datos.	Optimizar las consultas y repetir las mediciones. Distinguir latencia SQL de latencia HTTP y definir la carga normal representativa.
Completar una reserva en menos de 2 segundos desde el clic hasta la confirmación.	Carga operativa de reservas y consultas de disponibilidad.	Se reportan pruebas del módulo y problemas en consultas con grandes volúmenes. No se presenta una medición completa desde el clic hasta la confirmación.	Instrumentar el flujo completo y medirlo bajo la carga prevista.
Evitar tareas y reservas duplicadas.	Prueba de colisiones con 40 solicitudes simultáneas al mismo salón y horario.	No cumple para reservas en el escenario probado: se aceptaron múltiples solicitudes. No se presenta un resultado equivalente para duplicados de tareas.	Corregir la validación concurrente, comprobar el estado final de la BD y repetir la prueba.
Control atómico de reservas ante solicitudes simultáneas.	Escenario relacionado con PE-RES-01.	Incumplimiento detectado: la validación previa a la inserción permitió una condición de carrera.	Asegurar que la comprobación de disponibilidad y la creación de la reserva mantengan exclusión efectiva bajo concurrencia.
Soportar entre 100 y 500 usuarios registrados.	Pruebas de carga y volumen sintético.	Se amplió considerablemente el volumen de datos probado, pero no se documenta una evaluación específica con 100–500 cuentas registradas.	Preparar un conjunto representativo de usuarios y comprobar su impacto.
Soportar 3–5 administrativos concurrentes y picos de 30–50 usuarios.	Pruebas de carga operativa, picos y resistencia prolongada con k6.	El informe registra ejecución de estos escenarios y recuperación después de los picos. No presenta todos los parámetros necesarios para contrastarlos con los rangos del RNF.	Adjuntar usuarios concurrentes, duración, endpoints, errores y percentiles de cada ejecución.
Mantener rendimiento al crecer los datos.	Sysbench con hasta 100.000 reservas y 1.000.000 de asociaciones.	Las inserciones reportaron aproximadamente 31.400 transacciones/s, con p95	Optimizar y verificar los planes de ejecución; repetir las consultas con los mismos conjuntos de datos.

de 0,84 ms. Las consultas complejas se degradaron en las escalas mayores.

Mantenibilidad infraestructura y confiabilidad

Requisito no funcional	Pruebas o trabajo realizado	Resultado actualizado	Verificación pendiente
Arquitectura modular y por capas, con cambios limitados al actualizar el sistema.	Plan de refactorización y tratamiento de deuda técnica.	Se documentan mejoras y deuda reprogramada. Esto evidencia trabajo de mantenibilidad, pero no verifica el límite del 20 % de código modificado.	Definir cómo medir ese porcentaje y revisar las dependencias entre capas.
Documentación técnica actualizada del 100 % del código.	Documentación del sprint, pruebas y refactorización.	Hay documentación técnica ampliada; no se presenta un inventario que demuestre cobertura documental del 100 %.	Contrastar módulos y componentes con su documentación.
Pruebas automatizadas con al menos 80 % de cobertura.	Suite del frontend y reporte de cobertura de la página 42.	Supera el 80 % en el código incluido en ese reporte: 99,82 % de sentencias, 97,4 % de ramas y 100 % de funciones y líneas.	Confirmar archivos incluidos y exclusiones; aportar cobertura del backend para evaluar el requisito sobre todo el proyecto.
Ejecución automática de controles de calidad.	CI de build, lint y pruebas del frontend; pruebas unitarias del backend.	La captura muestra dos checks exitosos. La captura de la suite muestra 177 pruebas aprobadas.	Corregir la discrepancia con el texto, que menciona 174 pruebas, y conservar la referencia de la ejecución.
Servidor de BD con al menos 10 GB de almacenamiento.	Medición del tamaño de datos e índices en las pruebas de volumen.	El informe señala un consumo inferior a 500 MB para el conjunto probado. Esto mide ocupación, no capacidad disponible del servidor.	Verificar capacidad asignada y espacio libre en el servidor.
Crecimiento de 2–5 GB anuales y escalabilidad horizontal.	Pruebas con tres escalas de datos.	Se evaluó crecimiento del volumen y se identificaron límites de rendimiento. No se demuestra una proyección anual ni escalado entre varias instancias.	Medir crecimiento representativo y ejecutar una prueba de escalabilidad horizontal.
Disponibilidad de 90–95 % durante los horarios definidos.	Pruebas de picos y carga sostenida.	El informe registra continuidad y recuperación en los escenarios probados. No incluye medición de	Definir periodo y horarios; calcular disponibilidad con registros de monitoreo.

		disponibilidad durante un periodo representativo.	
Respallos periódicos y recuperación de al menos 90 % de la información.	No se presenta una ejecución de restauración en las evidencias revisadas.	Pendiente de evidencia de recuperación. La estabilidad bajo carga no demuestra recuperación desde respaldos.	Restaurar una copia en un entorno aislado y comprobar porcentaje recuperado e integridad.

Portabilidad integraciones y soporte al usuario

Requisito no funcional	Pruebas o trabajo relacionado	Resultado actualizado	Verificación pendiente
Compatibilidad con Chrome, Firefox y Safari.	Pruebas automatizadas del frontend y compatibilidad prevista en el plan.	Las pruebas de componentes no equivalen a ejecuciones en cada navegador. El propio informe identifica pruebas multidispositivo pendientes.	Completar una matriz de navegadores y versiones.
Adaptación a resoluciones y funcionamiento en Android e iOS.	UX móvil y pruebas de interacción del frontend.	No se presentan resultados suficientes por sistema operativo, resolución y dispositivo.	Registrar pruebas reales en Android e iOS y distintos tamaños de pantalla.
Acceso mediante tecnologías web estándar, sin instalaciones adicionales.	Desarrollo y pruebas de la aplicación web.	Existe funcionamiento documentado de la aplicación en navegador, pero no una comprobación específica en perfiles limpios y distintos equipos.	Verificar los flujos principales sin extensiones o complementos requeridos.
Funcionamiento en distintas redes y con datos móviles.	Pruebas remotas y simulación de fallos de red en el frontend.	Se documenta manejo automatizado de errores de red. No se demuestra funcionamiento en todas las redes previstas.	Probar acceso, desconexión y recuperación con redes reales.
Integración con WhatsApp y Google Calendar.	No se presentan pruebas directas de estas integraciones.	Pendiente de evidencia específica. Las notificaciones internas y el calendario propio no acreditan esas integraciones.	Comprobar entrega externa y sincronización de eventos y horarios.
Persistencia correcta en una base de datos SQL centralizada.	Pruebas de reservas, carga concurrente y Sysbench sobre MariaDB.	Hay evidencia de pruebas contra el motor SQL. La creación de reservas duplicadas demuestra una brecha de integridad bajo concurrencia.	Corregir la condición de carrera y verificar persistencia e integridad de los flujos afectados.
Ayuda integrada, tutorial y seguimiento de soporte.	Documentación y gestión de	No se presentan resultados específicos sobre comprensión de	Incorporar registros de uso y seguimiento de incidencias.

incidencias del sprint.

ayudas, tutoriales o resolución de solicitudes de soporte.

Seguridad privacidad y control de acceso

Requisito o control relacionado	Pruebas o trabajo realizado	Resultado actualizado	Verificación o acción pendiente
Autenticación mediante usuario y contraseña.	Login válido, login inválido y consulta autenticada.	Se obtuvieron respuestas 200 con credenciales válidas y 401 con credenciales incorrectas.	Completar las variantes de sesión y acceso no cubiertas por estas ejecuciones.
Bloqueo de 5 minutos tras 5 intentos fallidos.	Tarea de rate limiting reportada como finalizada.	La finalización de la tarea no demuestra que coincida exactamente con el bloqueo por usuario y la duración exigidos.	Ejecutar y documentar la secuencia de cinco fallos, el bloqueo y su vencimiento.
Contraseña con longitud y complejidad mínimas.	Pruebas de formularios y validaciones del frontend.	No se identifica una matriz completa de las reglas de contraseña requeridas.	Probar cada regla y sus límites en registro y cambio de contraseña, también en backend.
Acceso a datos solo para miembros autorizados.	Trabajo de RBAC en grupos y tareas; pruebas de roles del frontend; CI del backend.	El informe documenta avances en autorización. Las pruebas visuales de roles no sustituyen la comprobación de permisos en la API.	Vincular resultados negativos por rol, operación y recurso.
Protección de secretos y configuración.	DT-07 y DT-08 reportadas como abordadas.	Se documentan eliminación de secretos de respaldo y validación de variables obligatorias.	Asociar evidencia de arranque fallido ante variables ausentes y verificar el despliegue correspondiente.
Protección de la cookie de refresh.	PS-SEG-09 manual y automatizada.	No cumple en la ejecución registrada: falta Secure.	Corregir y repetir el login para comprobar los tres atributos exigidos.
Ausencia de cuentas de prueba con contraseñas conocidas.	PS-SEG-10.	No cumple: una cuenta de prueba permitió acceder a un endpoint protegido.	Retirar o actualizar credenciales e invalidar sesiones; repetir la prueba.
Aislamiento de MariaDB.	PS-SEG-11, tres conexiones externas a TCP 3306.	No cumple: el servicio aceptó conexiones y anunció MariaDB.	Restringir la exposición y verificar tanto el bloqueo externo como el acceso interno autorizado.
Headers seguros y manejo de errores.	PS-SEG-12 A y B.	12A no cumple por headers ausentes. 12B cumple en 401 y 404, sin información interna.	Verificar las correcciones en el despliegue y evaluar 500 en un entorno controlado.

Política CORS por entorno.	Revisión de configuración y solicitud con origen localhost.	La ejecución registrada permitió localhost:5173, aunque mantuvo el rechazo por falta de autenticación.	Repetir la prueba sobre la versión que incorpora la corrección de CORS.
Auditoría de acciones clave.	No se presenta una verificación específica de los registros de auditoría.	Pendiente de evidencia.	Ejecutar acciones y contrastar usuario, operación y fecha con los registros generados.

Restricciones culturales legales y lectura de resultados

Requisito no funcional	Prueba relacionada	Resultado actualizado	Verificación pendiente
Contenido visual coherente con los principios de la iglesia.	Revisión por representantes y demostración a usuarios finales.	La demostración puede aportar retroalimentación, pero no se presenta una aprobación específica de todo el contenido.	Registrar la validación de los representantes.
Ausencia de organizaciones e integraciones externas no autorizadas.	Revisión de contenido e integraciones.	No se presenta un registro específico de autorizaciones.	Revisar menciones, servicios externos y aprobaciones.
Horarios y eventos adaptables a actividades religiosas.	Implementación de reservas, calendario y portal público; pruebas de sus flujos.	Existe trabajo funcional relacionado. La adecuación a las actividades religiosas requiere validación del cliente.	Documentar escenarios y aceptación de los representantes.
Políticas de privacidad y términos visibles.	Revisión de contenido y navegación.	El informe los mantiene como aspectos pendientes de revisión.	Comprobar acceso, visibilidad y contenido aplicable.
Cumplimiento legal y de estándares como ISO 27001.	Pruebas de seguridad como evidencia técnica de apoyo.	Las pruebas identifican riesgos, pero no acreditan cumplimiento normativo ni certificación.	Definir alcance y controles aplicables. ISO 27001 corresponde a gestión de seguridad de la información, no a una auditoría de accesibilidad.

Refactorización

Objetivo y alcance del plan

La refactorización del Sprint 8 se orienta a disminuir primero los riesgos de seguridad e integración identificados en la matriz de deuda técnica del Sprint 7, sin modificar el comportamiento funcional esperado. El plan asigna cada deuda a un sprint y vincula las tareas actuales con responsables, estimaciones y criterios de verificación.

La planificación toma como base DT-01 a DT-12 del informe anterior y añade AT-05, hallazgo relacionado con la separación entre los entornos de desarrollo y producción. DT-01 ya fue completada durante el Sprint 7 y no requiere una nueva tarea.

Plan de refactorización del Sprint 8 y siguientes

La capacidad del Sprint 8 se concentra en seguridad, integración continua y pruebas. Las deudas de mantenibilidad que no caben en los 81 puntos planificados se trasladan al Sprint 9 con un orden explícito, evitando que queden sin seguimiento.

Matriz de deuda técnica y planificación

ID y estado	Deuda y área	Evidencia e impacto	Técnica y momento
DT-04 Abordada	Automatización e integración continua.	El CI no ejecutaba lint ni pruebas del frontend y no funcionaba como verificación obligatoria.	Sprint 8: CI con build, lint y pruebas de frontend. Responsable: Pedro Caso.
DT-05 Abordada	Control de acceso en rutas de grupos.	Las operaciones de escritura solo exigían autenticación y no validaban el rol.	Sprint 8: RBAC en rutas de grupos y pruebas negativas por rol. Responsable: Diego Calderón.
DT-06 Abordada	Control de acceso en mutaciones de tareas.	Crear, eliminar o asignar tareas carecía de una política uniforme de rol o propiedad.	Sprint 8: RBAC en rutas de tareas y centralización de la autorización. Responsable: Diego Calderón.
DT-07 Abordada	Secretos JWT de respaldo.	El código contenía valores predeterminados que podían permitir firmar	Sprint 8: eliminar respaldos y centralizar la lectura de secretos.

		tokens con secretos predecibles.	Responsable: Diego Calderón.
DT-08 Abordada	Validación de variables de entorno.	La ausencia de variables obligatorias se detectaba tardíamente durante la ejecución.	Sprint 8: validar la configuración y detener el arranque si falta un valor. Misma tarea de DT-07.
DT-12 Abordada	Cobertura de pruebas del frontend.	La cobertura se limitaba a un archivo de pruebas y no incluía páginas, formularios ni hooks.	Sprint 8: ampliar cobertura y ejecutarla en CI. Responsable: Hugo Méndez.
DT-02 Pendiente	Tipado y consistencia.	Persisten errores de configuración de Vitest y usos de any en reserva.controller.ts.	Sprint 9: completar tipado y estabilización del frontend.
DT-03 Pendiente	Estados y componentes del frontend.	Persisten comparaciones directas de estados y estilos estáticos en componentes compartidos.	Sprint 9: usar ESTADOS_RESERVA y trasladar estilos a módulos CSS.
DT-09 Pendiente	Transacciones SQL repetidas.	Cuatro controladores repiten conexión, inicio, commit, rollback y liberación.	Sprint 9: extraer withTransaction y migrar los controladores de forma incremental.
DT-10 Pendiente	Estado de autenticación en componentes.	Siete componentes leen y analizan el usuario directamente desde localStorage.	Sprint 9: consumir useAuth() y añadir pruebas de actualización de sesión.
DT-11 Pendiente	Tipos de dominio duplicados.	Ocho archivos mantienen interfaces locales que contradicen o duplican types/index.ts.	Sprint 9: consolidar Reserva, Tarea, Ministro y Persona antes de ampliar esos módulos.
AT-05 Pendiente	Separación de desarrollo y producción.	Los contenedores usan servidores de desarrollo y el despliegue provoca indisponibilidad al reconstruir.	Sprint 9: crear imágenes de producción y un despliegue sin downtime.

Priorización y plan de ejecución

Sprint 8 - seguridad antes del siguiente despliegue. Se atienden DT-05, DT-06,

DT-07 y DT-08 mediante controles de acceso por rol, eliminación de secretos de respaldo y validación temprana de la configuración.

Sprint 8 - calidad de integración. Se atienden DT-04 y DT-12 mediante un flujo de CI que ejecute build, lint y pruebas del frontend, junto con una ampliación de la cobertura automatizada.

Sprint 9 - mantenibilidad. Se reprograman DT-02, DT-03, DT-09 y DT-11; DT-10 permanece en el Sprint 9 conforme al plan original. El orden propuesto es DT-02, DT-09, DT-03, DT-11 y DT-10, con pruebas de regresión en cada cambio.

AT-05 - despliegue de producción. Se mantiene fuera del alcance del Sprint 8 por capacidad. En el Sprint 9 se deberán crear imágenes de producción para backend y frontend y reemplazar el despliegue con downtime por una estrategia de actualización controlada.

Refactorización abordada durante el Sprint 8

El Sprint Backlog incluye seis deudas de la matriz: DT-04, DT-05, DT-06, DT-07, DT-08 y DT-12. Estas tareas cubren las prioridades de seguridad e integración definidas en el informe del Sprint 7 y tienen responsables y fechas de terminación dentro del Sprint 8.

El cierre de cada deuda requiere evidencia en el repositorio y en Jira: pruebas de autorización por rol, fallo de arranque ante variables ausentes, ausencia de secretos de respaldo, ejecución automática del build, lint y pruebas, y resultados verificables de la nueva cobertura del frontend.

Deuda técnica reprogramada

DT-02, DT-03, DT-09 y DT-11 estaban previstas originalmente para el Sprint 8, pero se trasladan al Sprint 9 para no desplazar HU-30, HU-31 y HU-32 ni las correcciones críticas DT-05 a DT-08. DT-10 ya estaba planificada para el Sprint 9.

AT-05 también queda programada para el Sprint 9. Su alcance comprende separar las configuraciones de desarrollo y producción, generar artefactos optimizados y evitar la indisponibilidad causada por reconstruir todos los servicios durante cada despliegue.

Cada deuda pendiente se registrará como una tarea independiente en Jira, con responsable, estimación, criterio de aceptación y evidencia técnica. Su estado se revisará durante la planificación y retrospectiva del Sprint 9.

Resultado y deuda remanente

Con este plan, el Sprint 8 reduce primero los riesgos de seguridad, configuración e integración. La deuda remanente queda identificada y calendarizada para el Sprint 9: DT-02, DT-03, DT-09, DT-10, DT-11 y AT-05. La reprogramación deberá confirmarse al iniciar ese sprint y ajustarse únicamente si cambia la prioridad del producto o aparece un riesgo crítico.

Ejecución del Presupuesto

Con corte a la fecha, se compara el avance real del equipo contra el presupuesto y el tiempo total estimados por el método Bottom-Up (WBS), para dar seguimiento a la ejecución del proyecto.

Porcentaje de tiempo consumido

Valorado las 949 horas acumuladas a la misma tarifa de mercado utilizada en la estimación (Q300/h), el costo real de desarrollo a la fecha es de Q284,700.00. Sumando la porción proporcional de infraestructura operativa consumida, el gasto total acumulado es de Q287,596.82.

Concepto	Valor
Presupuesto total estimado (WBS)	Q 320,673.40
Gasto real acumulado a la fecha	Q 287,596.82
Presupuesto restante	Q 33,076.58
% DEL PRESUPUESTO GASTADO	89.69%

Se ha gastado hasta el momento el 89.69% del presupuesto total estimado (Q 287,596.82 de Q 320,673.40).

Porcentaje del costo ejecutado

El proyecto estima un total de 1,054 horas de desarrollo según el método Bottom-Up. A la fecha, incluyendo el Sprint 8 recién cerrado, se han consumido 949 horas.

Concepto	Valor
Tiempo total estimado (WBS)	1,054 h
Tiempo real consumido a la fecha	949 h
Tiempo restante estimado	105 h
% DEL TIEMPO CONSUMIDO	90.04%

Se ha consumido hasta el momento el 90.04% del tiempo total estimado del proyecto (949 h de 1,054 h).

Pruebas Unitarias

En el entorno Frontend, se implementaron un total de 174 pruebas unitarias utilizando React Testing Library, logrando alcanzar el 100% de en los componentes visuales, páginas, utilidades y custom hooks. Los escenarios más relevantes evaluados y su impacto son:

1. Pruebas de Frontend (Componentes de UI e Interacción Dinámica):
 - a. Renderizado condicional y estados de error: Verifica que componentes genéricos (Spinner, EmptyState, ErrorBoundary) se muestren correctamente bajo distintos escenarios y eviten que un error en el árbol de componentes colapse la aplicación entera.
 - b. Accesibilidad y eventos de teclado: Asegura que elementos interactivos como Modales puedan ser cerrados o manipulados mediante eventos del teclado (por ejemplo, presionar "Escape"), respetando los bloqueos cuando hay acciones en proceso.
2. Pruebas de Frontend (Formularios y Validaciones):
 - a. Bloqueo preventivo de envío: Valida que formularios clave (como EnviarNotificacionForm o ModalExcusaAsistencia) evalúen correctamente las longitudes mínimas/máximas de los campos y verifiquen selecciones obligatorias antes de permitir una petición a la API.
 - b. Manejo de selecciones masivas (Checkboxes): Garantiza matemáticamente que los botones de "Seleccionar todos" o "Quitar todos" funcionen impecablemente sobre listas agrupadas y que los estados parciales se mantengan sincronizados con la vista.
3. Pruebas de Frontend (Integración de APIs y Custom Hooks):
 - a. Manejo silencioso de errores de red: Simula caídas de red o rechazos de API durante operaciones asíncronas (como confirmar o cancelar una

asistencia), certificando que la UI capture el error, no interrumpa el flujo del usuario y retorne a su estado original sin desincronización.

- b. Sincronización de estados locales: Prueba que los hooks (`useNotificaciones`, `usePapelera`) reflejen inmediatamente los cambios (como mover un elemento a la papelera) en las variables reactivas de la UI sin necesidad de recargar la ventana.

4. Pruebas de Frontend (Utilidades de Negocio y Seguridad Visual):

- a. Jerarquía de Roles de Usuario: Verifica que funciones críticas, como `usuarioTieneRol`, procesen correctamente los datos almacenados en `cache/localStorage`, inhabilitando o bloqueando preventivamente menús y botones según el rol (Administrador, Sacerdote, Ministro) que posea la sesión activa.
- b. Formateo de Fechas: Valida la consistencia en el despliegue de horarios y fechas, asegurando que las transformaciones locales sean exactas en distintas zonas horarias.

Test Files **18 passed** (18)
 Tests **177 passed** (177)
 Start at 03:34:11
 Duration 4.57s (transform 1.64s, setup 1.69s, import 3.59s, tests 6.00s, environment 16.43s)

% Coverage report from v8

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	99.82	97.4	100	100	
api	100	100	100	100	
ausencias.ts	100	100	100	100	
auth.ts	100	100	100	100	
client.ts	100	100	100	100	
contacto.ts	100	100	100	100	
components/layout	100	100	100	100	
Sidebar.module.css	0	0	0	0	
Sidebar.tsx	100	100	100	100	
components/ui	100	100	100	100	
Badge.module.css	0	0	0	0	
Badge.tsx	100	100	100	100	
Btn.module.css	0	0	0	0	
Btn.tsx	100	100	100	100	
Card.module.css	0	0	0	0	
Card.tsx	100	100	100	100	
EmptyState.module.css	0	0	0	0	
EmptyState.tsx	100	100	100	100	
ErrorBoundary.tsx	100	100	100	100	
ErrorMessage.module.css	0	0	0	0	
ErrorMessage.tsx	100	100	100	100	
ErrorState.module.css	0	0	0	0	
ErrorState.tsx	100	100	100	100	
Field.module.css	0	0	0	0	
Field.tsx	100	100	100	100	
LoadingState.module.css	0	0	0	0	
LoadingState.tsx	100	100	100	100	
Modal.module.css	0	0	0	0	
Modal.tsx	100	100	100	100	
PageHeader.module.css	0	0	0	0	
PageHeader.tsx	100	100	100	100	
Spinner.module.css	0	0	0	0	
Spinner.tsx	100	100	100	100	
context	100	100	100	100	
AuthContext.tsx	100	100	100	100	
modules/ausencias/components	100	100	100	100	
NotificarAusenciaForm.module.css	0	0	0	0	
NotificarAusenciaForm.tsx	100	100	100	100	
modules/notificaciones/components	99.35	98.47	100	100	
EnviadaRow.module.css	0	0	0	0	
EnviadaRow.tsx	100	95.45	100	100	50
EnviarNotificacionForm.module.css	0	0	0	0	
EnviarNotificacionForm.tsx	100	100	100	100	
ModalExcusaAsistencia.module.css	0	0	0	0	
ModalExcusaAsistencia.tsx	96.87	95.45	100	100	35
NotificacionRow.module.css	0	0	0	0	
NotificacionRow.tsx	100	100	100	100	
PapeleraRow.tsx	100	100	100	100	
modules/notificaciones/hooks	100	85.71	100	100	
useNotificaciones.ts	100	83.33	100	100	92-103
useNotificacionesEnviadas.ts	100	100	100	100	
usePapelera.ts	100	100	100	100	
pages/ausencias	100	100	100	100	
AusenciasPage.module.css	0	0	0	0	
AusenciasPage.tsx	100	100	100	100	
pages/landing	100	100	100	100	
ContactoForm.tsx	100	100	100	100	
pages/notificaciones	100	90.16	100	100	
NotificacionesPage.module.css	0	0	0	0	
NotificacionesPage.tsx	100	90.16	100	100	145,187-194,239
styles	0	0	0	0	
Form.module.css	0	0	0	0	
utils	100	100	100	100	
date.ts	100	100	100	100	
roles.ts	100	100	100	100	

All checks have passed

2 successful checks

✓  CI / Frontend Build, Lint and Tests (pull_request) ... [Details](#)

✓  CI / Backend Unit Tests (pull_request) Successful ... [Details](#)

Pruebas de Carga y Estrés

Herramienta utilizada

Para la implementación de las pruebas de carga y estrés en el proyecto, se seleccionó K6 como herramienta de pruebas principal.

Justificación

Se seleccionó K6 como la herramienta ideal para ejecutar pruebas de carga y estrés por las siguientes razones: los scripts de prueba se escriben en JavaScript, dado a que ya se utiliza JS/TS para el backend y frontend, la curva de aprendizaje es mínima y se pueden mantener y extender las pruebas fácilmente. Por otro lado, a diferencia de herramientas antiguas basadas en Java o en Node puro, k6 utiliza Go, esto le permite generar altísimas cargas concurrentes consumiendo muy pocos recursos de memoria y CPU en la máquina local o el contenedor de pruebas. K6 se diseñó pensando en la automatización moderna, su fácil ejecución mediante un contenedor de Docker y comandos de terminal simplifica enormemente su integración futura en pipelines de GitHub Actions. Por último, K6 es excepcionalmente bueno para simular peticiones HTTP(S) hacia APIs REST, manejar cookies, tokens de sesión y evaluar métricas precisas

Resultados de las pruebas

Se amplió la suite de pruebas incorporando escenarios extremos de rendimiento y asaltos concurrentes para evaluar la robustez transaccional y la resistencia del sistema:

- Carga Operativa de Reservas y Calendario: Se evaluó la creación de reservas y la paginación masiva sobre el calendario mensual. El sistema mantuvo tiempos de respuesta consistentes bajo estrés sin degradación severa en las lecturas complejas de fecha.

- **Prueba de Colisiones Concurrentes (Condiciones de Carrera):** Se enviaron 40 peticiones instantáneas e idénticas intentando reservar exactamente el mismo salón, día y hora. La prueba expuso un fallo crítico en la lógica de backend: se aceptaron múltiples reservas paralelas (HTTP 201) en lugar de rechazar todas menos una. Esto reveló que la validación en código ocurre antes de la inserción real, abriendo una ventana de microsegundos donde varias peticiones asumen que el espacio está libre.
- **Pruebas Spike (Aluviones Súbitos de Tráfico):** Se bombardeó el servidor con picos anómalos de Usuarios Virtuales en ventanas de pocos segundos. Se confirmó que, gracias a la gestión de colas de Express y la base de datos local, el servidor sobrevive sin reiniciarse (caerse), reanudando su latencia original una vez que pasa la ola de peticiones.
- **Pruebas Soak (Resistencia Prolongada):** Se inyectó un tráfico constante y mantenido sobre el backend simulando horas de operación ininterrumpida. Validó la solidez de la infraestructura, confirmando que no existen fugas de memoria (memory leaks) ni conexiones estancadas (zombies) hacia MariaDB.

Plan de mitigación adicional (Transacciones y Concurrencia)

Para cerrar las brechas expuestas por los nuevos escenarios, se requieren las siguientes intervenciones:

- **Solución a las colisiones de reservas (Doble Agendamiento):**
 - **Pessimistic Locking:** Refactorizar el endpoint de creación de reservas integrando la validación y la inserción dentro de una única transacción con un bloqueo a nivel de fila de base de datos (`'SELECT ... FOR UPDATE'`).

- Constraints: Apoyarse directamente en restricciones dentro del esquema SQL de MariaDB para que el propio motor aborte matemáticamente cualquier transacción que solape un mismo `espacio\_id` en una misma franja horaria.

Pruebas de volúmenes, inundación y requisitos no funcionales

Herramienta y Metodología

Para realizar la validación de volumen y resistencia de la base de datos MariaDB, se implementó un entorno aislado utilizando Docker y Sysbench (mediante scripts personalizados en lenguaje Lua). Sysbench fue seleccionado por su capacidad de generar carga multihilo directamente sobre el motor SQL, permitiéndonos evaluar el comportamiento nativo de MariaDB independiente de los cuellos de botella del servidor backend de Node.js.

Se estructuraron tres escalas de volumen sintético para analizar la degradación progresiva de los datos:

- Small (Pequeña): 1,000 reservas y 10,000 asociaciones en `persona\_notificacion`.
- Medium (Mediana): 10,000 reservas y 100,000 asociaciones.
- Large (Grande): 100,000 reservas y 1,000,000 asociaciones.

Resultados de las pruebas

1. Inserciones Masivas / Inundación (PI-01) El motor de MariaDB demostró ser sumamente robusto para la ingesta de datos. Durante las pruebas de Flood bajo las tres escalas, el rendimiento se mantuvo estable con ~31,400 transacciones por segundo (TPS). La latencia del percentil 95 (p95) se mantuvo en 0.84 ms a pesar de alcanzar el millón de registros, comprobando que no hay pérdida de datos ni contención transaccional durante las escrituras.

2. Consultas de Volumen (PV-01 y PV-02) Se identificó una degradación severa del rendimiento en lecturas complejas conforme crecía el volumen de datos:

- Escala Small: Latencia p95 de 4.57 ms (17,000 consultas/s).
- Escala Medium: Latencia p95 de 4,358 ms / ~4.3 segundo.
- Escala Large: Latencia p95 de **8,638 ms / ~8.6 segundos.

Problemas y Soluciones Propuestas

- Problema: Incumplimiento del SLA de latencia en tablas grandes (Full Table Scans).
 - Causa: Las consultas críticas, como el cálculo de disponibilidad de espacios (`'CASE WHEN EXISTS...'`) y el histórico de notificaciones (`'JOIN persona_notificacion ORDER BY fecha DESC'`), carecen de índices compuestos que respalden los filtros.
 - Solución: Crear los siguientes índices en la base de datos:


```
CREATE INDEX idx_reserva_disponibilidad ON reserva (espacio_id, fecha, estado_reserva_id);
```

```
CREATE INDEX idx_notificacion_orden ON notificacion (fecha DESC);
```

Comparación con Pruebas Anteriores (Carga/Estrés k6)

A diferencia de las pruebas HTTP con k6, donde el cuello de botella residía en la capacidad de CPU del backend (cálculos de `'bcrypt'`) y el Event Loop de Node.js, estas pruebas de volumen aislaron el motor de base de datos. En k6 con volumen base (1,000 registros), la base de datos MariaDB lograba p95 de ~4 ms y se comportó estable. En Sysbench, pudimos observar el futuro del sistema: si la parroquia acumula registros históricos (>100k), la base de datos se convertirá en el nuevo cuello de botella principal, degradando los tiempos de respuesta a casi 9 segundos.

Grado de Cumplimiento de Requisitos No Funcionales

Requisito original	Criterio de aceptación	Prueba y evidencia	Estado	Acción Pendiente
Tener un tiempo de respuesta menor a 2 segundos al realizar reserva.	Latencia máxima < 2s	Consultas PV-01 superaron los 8 segundos en escala Large.	Incumplido (A gran escala)	Implementar índices compuestos en MariaDB para garantizar $O(\log N)$.
El sistema debe de garantizar una precisión, impidiendo que se repitan tareas y salones reservados	Validación transaccional bajo alta concurrencia	Prueba Flood (Sysbench) e historial de k6 (PE-RES-01).	Parcialmente Cumplido	Reforzar las llaves únicas compuestas en la tabla reserva para forzar bloqueos a nivel motor SQL y mitigar condiciones de carrera.
Si el sistema presenta algún error, deberá mostrar claramente de que y por qué fue el error.	Logs y rollback seguro	Logs capturados durante la inserción masiva.	Cumplido	Ninguna. MariaDB emite los códigos y realiza rollbacks implícitos.
El servidor debe disponer de al menos 10 GB de almacenamiento.	Control del crecimiento de disco.	1 millón de registros ocupó menos de 500 MB en disco (datos + índices).	Cumplido	Monitoreo a largo plazo.

Calidad de los Requisitos y Pruebas Pendientes

Análisis del Diseño de Requisitos

Los requisitos planteados en los requisitos no funcionales presentan áreas de mejora.

El SLA de "tiempo de respuesta menor a 2 segundos" es un estándar aceptable, pero no detalla la carga base ni el percentil a medir (ej. p95 vs promedio). Requisitos como "El sistema debe ser colorido y atractivo" son ambiguos y difíciles de testear de forma objetiva sin métricas claras de UX/UI.

Pruebas Faltantes para el 100% de cobertura no funcional:

1. Pruebas de Seguridad y Penetración: Validación de ataques contra contraseñas e inyección SQL.
2. Pruebas Visuales Multi-dispositivo (Portabilidad): Comprobación real en Safari, Firefox y dispositivos móviles con resoluciones atípicas.
3. Auditoría de Accesibilidad Legal (ISO 27001): Revisión de privacidad de datos y visibilidad de términos y condiciones.

Almacenamiento y Servicio de Fotografías Reales de Espacios

Actualmente, los salones y áreas físicas del sistema (`Templo Principal`, `Salón Parroquial`, `Salas de Catequesis`, `Capilla Lateral`, `Patio Central`) no cuentan con fotografías reales provistas por el cliente. Para mitigar el impacto visual en el producto, la interfaz implementa un marcador de posición (*placeholder*) estilizado mediante CSS puro en los componentes `EspacioCard.tsx` y `EspacioDetallePage.tsx`

Hasta este sprint, el proyecto nunca ha manejado archivos multimedia ni subida/servicio de activos estáticos generados por usuarios o clientes. Ante la futura entrega de las fotos reales por parte de la parroquia, se realizó una investigación arquitectónica para

determinar la solución técnica idónea que garantice escalabilidad, seguridad y preservación de los Requisitos No Funcionales (NFRs).

Evaluación de Opciones Tecnológicas

Se evaluaron varios patrones arquitectónicos para el alojamiento y servicio de activos visuales de los cuales se eligieron 2:

- Opción A: Almacenamiento Local en Volumen Docker Persistente + Servicio Estático (Express / Nginx)
 - Mecanismo: Se define un volumen Docker persistente (`uploads_data:/app/uploads/espacios`) en el Droplet de producción. El backend Express sirve las imágenes mediante el middleware nativo `express.static('/uploads')` configurando cabeceras de caché HTTP agresivas (`Cache-Control: public, max-age=2592000, immutable`). En la base de datos se almacena únicamente la ruta relativa del archivo (`/uploads/espacios/templo-principal.webp`).
 - Ventajas:
 - Costo \$0: No requiere contratación de infraestructura externa ni tarjetas de crédito.
 - Soberanía y simplicidad: Los activos residen dentro del mismo servidor Droplet en DigitalOcean.
 - Compatibilidad directa con CSP: Cumple de inmediato con la directiva por defecto de Helmet (`'img-src 'self' data:'`), sin requerir aperturas de dominios externos. *
 - Desventajas:

- Consume ancho de banda y conexiones del servidor Node.js si no se interpone un proxy inverso (Nginx).
- Requiere incluir la carpeta de uploads en los procedimientos de respaldo rutinarios del servidor.
- Opción B: Plataforma Especializada de Medios y Optimización en Tiempo Real (Cloudinary / ImageKit)
 - Mecanismo: Servicio PaaS enfocado en procesamiento de imágenes. Permite servir imágenes mediante URLs dinámicas que aplican transformaciones automáticas de tamaño, calidad y formato al vuelo (``f_auto,q_auto,w_600``).
 - Ventajas:
 - Capa gratuita permanente: Cloudinary ofrece 25 créditos mensuales (~25,000 transformaciones/mes), sobrada para el volumen parroquial.
 - Cero dependencias pesadas en el servidor: No requiere compilar librerías nativas de C++ como ``sharp`` dentro del contenedor Alpine de Node.js para recortar o redimensionar fotos.
 - CDN integrada de alto rendimiento.**
 - Desventajas:
 - Dependencia de un tercero (Vendor lock-in ligero).
 - Requiere autorizar el dominio de la CDN en la cabecera ``Content-Security-Policy`` de Helmet en el backend (``res.cloudinary.com``).

Impacto en los Requisitos No Funcionales

1. Requisito de Almacenamiento (Disponer de al menos 10 GB): La parroquia dispone de 7 espacios físicos principales (``02_seeds.sql``). Asumiendo una proyección de crecimiento futuro a un máximo de 25 salones, cada fotografía

estandarizada en formato moderno WebP a resolución adecuada (1200×800 px para vista detalle y 400×260 px para tarjetas) tendrá un peso promedio de 120 KB.

- Cálculo de volumen proyectado:

$$VolumenTotal = 25espacios \times 120KB \approx 3,000KB = 3MB$$

- Conclusión: 3 MB representan únicamente el 0.03% de la cuota de 10 GB del servidor. El impacto sobre el almacenamiento en disco es prácticamente despreciable y cumple 100% con el NFR evaluado.
2. Requisito de Tiempo de Respuesta (< 2 segundos): Al configurar cabeceras ``Cache-Control: public, max-age=2592000, immutable`` y ``ETag``, el navegador descarga la imagen una única vez. Las visitas subsecuentes responden con código HTTP ``304 Not Modified`` o se leen directamente desde memoria caché local (``0 ms``), evitando saturar el ancho de banda del Droplet.
 3. Seguridad y Content Security Policy (CSP): En ``app/backend/src/app.ts``, la aplicación utiliza ``helmet()``, el cual por defecto restringe las fuentes de imágenes a ``img-src 'self' data:``.
 - El uso de la Opción A opera de manera nativa bajo ``'self'`` sin exponer la aplicación a riesgos de inyección de recursos externos ni alertas en navegadores estrictos.

Recomendación Técnica y Plan de Implementación

Para Sprints Futuros Para atender la entrega de fotografías por parte del cliente sin introducir deuda técnica ni incurrir en costos redundantes, se recomienda un enfoque pragmático en dos etapas:

- Etapa 1: Servicio Local Optimizado (Al recibir las fotos del cliente)

1. Ingesta y Formato: Convertir las fotografías proporcionadas por el cliente a formato `.webp` con compresión de calidad 80-85% (reducción del 30-40% respecto a JPEG).
 2. Migración de Base de Datos: Ejecutar una migración aditiva simple que agregue el campo opcional.
 3. Configuración en Backend (`app/backend/src/app.ts`): Exponer el directorio estático público protegido.
 4. Resiliencia en Frontend (Graceful Degradation): En `EspacioCard.tsx` y `EspacioDetallePage.tsx`, mantener el diseño del `photoSlot` actual como mecanismo de reserva (*fallback*). Si `foto_url` es nulo o la imagen física no responde (`onError`), el componente continúa mostrando armónicamente el marcador litúrgico actual sin quebrar la interfaz.
- Etapa 2: Escalabilidad a Nube / PaaS: Si en el futuro se incorpora una historia de usuario que permita a los administradores o sacerdotes subir y recortar fotografías directamente desde la interfaz gráfica web, se recomienda activar la integración con Cloudinary o Cloudflare R2, actualizando la directiva `img-src` de Helmet para autorizar el origen de la CDN externa.

Pruebas de Usabilidad con Usuarios Reales

Se realizaron sesiones de prueba con tres participantes reales, en las que se midió el tiempo de ejecución, la cantidad de errores y la percepción de dificultad de siete tareas representativas del sistema. Todas las tareas fueron completadas con éxito

Resultados por tarea

Participante	Tarea	Tiempos (s)	Errores	Resultado
P1	Crear una reserva	30	0	Completada
P2	Crear una reserva	28	0	Completada
P3	Crear una reserva	31	0	Completada

Percepción de dificultad

Cada participante calificó de 1 a 10 qué tanto le costó usar el sistema, donde 1 corresponde a “muy fácil” y 10 a “muy difícil”.

Participante	Dificultad	Comentario del participante
P1	1/10	Todo bien a la hora de crear la reserva
P2	1/10	Me pareció bien que en la sidebar exista un botón para crear una reserva
P3	2/10	Todo bien, ningún comentario

Hallazgos por participante

Participante 1

Completo la tarea asignada sin ayuda, en un lapso de 30 segundo, por lo que la tarea esta fácil de realizar y se redujo el tiempo de la última UX. Indicó que, por el momento, no cambiaría nada del sistema.

Participante 2

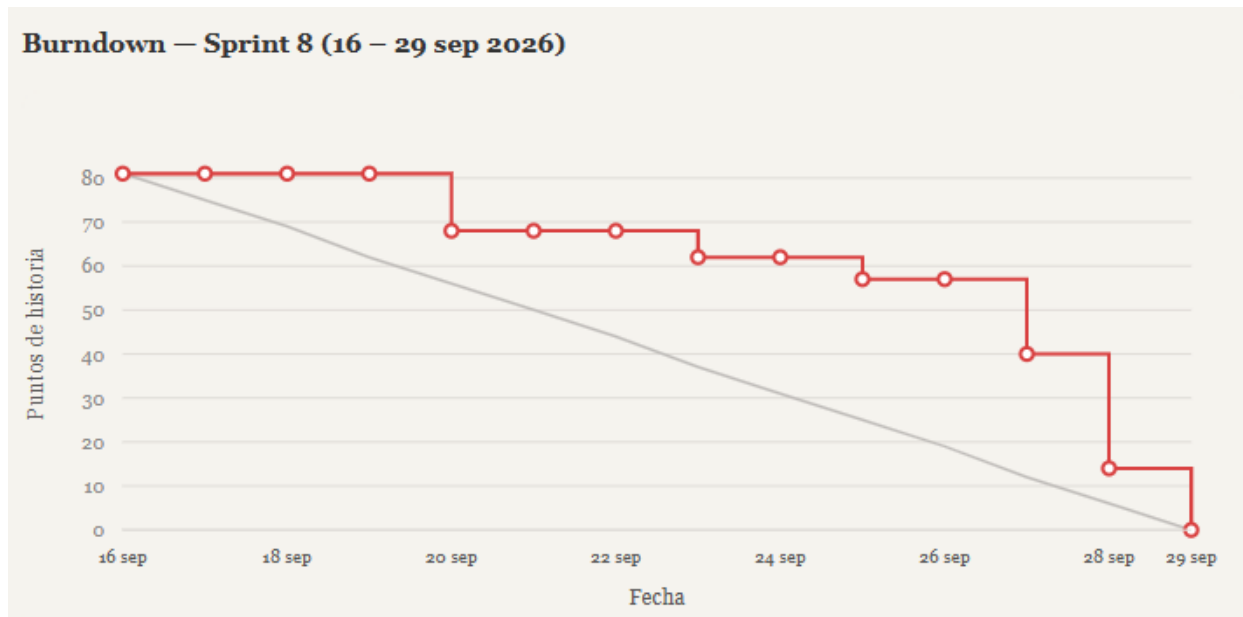
Fue el participante con menos tiempo un total de 28 segundo para poder realizar la reserva. Señaló que el mensaje que confirma que la reserva ya se realizó, se mira bien y que esta perfecto que te salga que fue lo que realizaste de reserva.

Participante 3

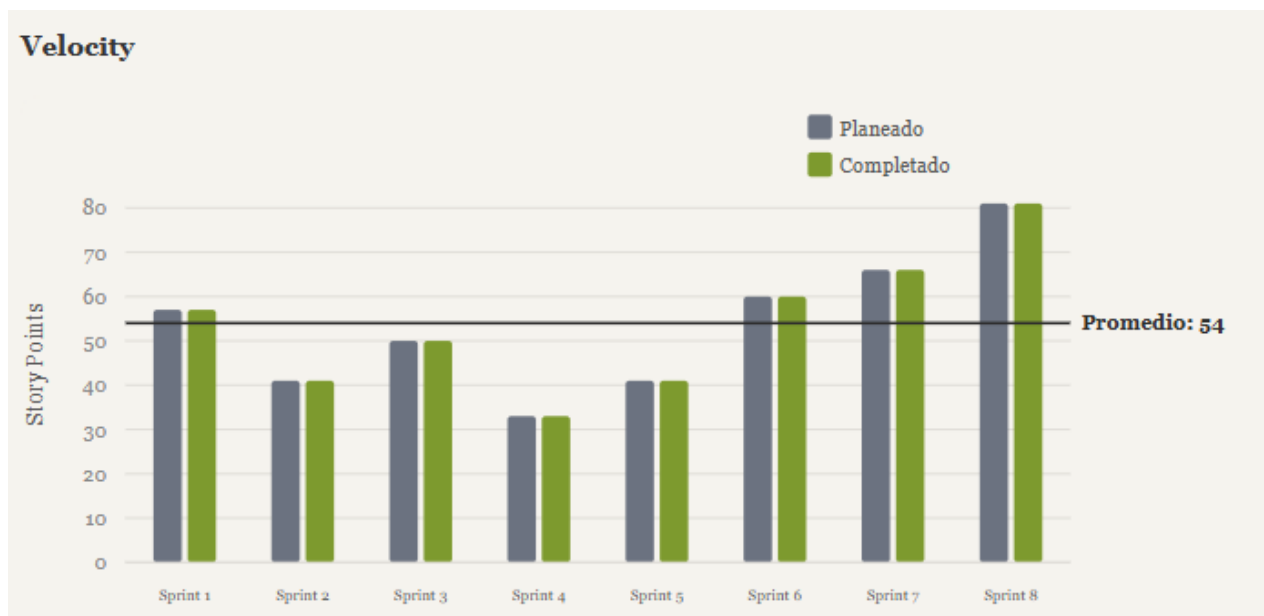
Obtuvo los mejores tiempos con 31 segundos para poder realizar una reserva. Le parece excelente que en la vista de mis reservas exista un botón que diga nueva reserva, para poder realizar la respectiva.

Métricas del Sprint

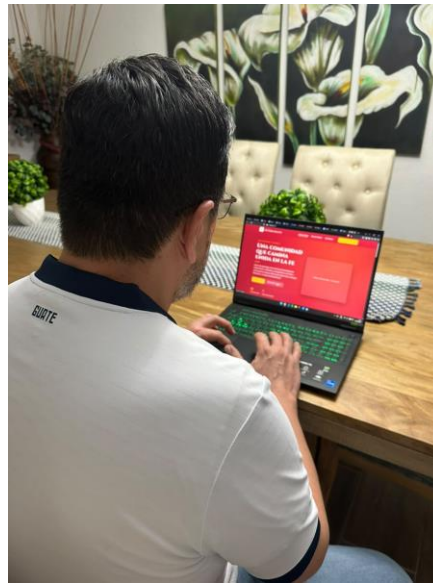
Burndown Chart



Velocity Chart



Retroalimentación de Product owners



Lo revisó todo y le parece que está muy bien. Solo tiene una duda puntual: al aceptar la primera solicitud de cambio de cambio ya que se le puede mandar a uno o más ministro el mismo cambio de turno, ¿el sistema rechaza de forma automática todas las demás? Fuera de ese detalle, todo queda aprobado de su lado.

Retrospectiva del Sprint

Aspectos positivos

Durante el Sprint 8 el equipo abordó de forma simultánea funcionalidades de producto, seguridad, refactorización, pruebas y documentación. Una mejora clara respecto a los sprints anteriores fue que la distribución de tareas se definió con suficiente tiempo de anticipación, estableciendo responsables, fechas de terminación y dependencias antes de concentrar el trabajo de desarrollo. Esto permitió iniciar el sprint con mayor claridad sobre el alcance y redujo la incertidumbre que había afectado planificaciones anteriores. También se avanzó en HU-30, HU-31 y HU-32, incorporando las notificaciones relacionadas con cambios de turno y periodos de ausencia, además del portal público para consultar

información parroquial sin iniciar sesión. En paralelo, se priorizaron riesgos técnicos relevantes mediante los controles de acceso por rol, la eliminación de secretos de respaldo, la validación de variables de entorno y el fortalecimiento de la integración continua y de las pruebas del frontend. Finalmente, el equipo mantuvo una visión integral del producto al complementar el desarrollo con pruebas de seguridad, volumen, experiencia de usuario, revisión del presupuesto y preparación de la demostración a usuarios finales.

Aspectos negativos

El principal problema del Sprint 8 fue el cumplimiento tardío del calendario. Varias tareas comenzaron o recibieron la mayor parte del trabajo después de la fecha de terminación definida y muy cerca del cierre del sprint. Esta concentración redujo el tiempo disponible para revisar integraciones, corregir defectos, actualizar Jira y preparar con calma las evidencias del informe. La situación también refleja que los 81 puntos planificados superaron la capacidad histórica del equipo, cuyo máximo anterior era de aproximadamente 66 puntos, por lo que el margen para absorber bloqueos o retrabajo fue limitado. Aunque la distribución asignaba fechas y responsables claros, el seguimiento diario no detectó con suficiente anticipación las tareas que se estaban retrasando y parte del trabajo terminó acumulándose en los últimos días.

Medidas a tomar para el siguiente sprint

Para el siguiente sprint se recomienda ajustar el alcance a la velocidad real del equipo y clasificar desde la planificación las tareas obligatorias, las deseables y las que pueden trasladarse sin afectar el incremento. Cada responsable deberá actualizar Jira durante el desarrollo y no únicamente al cierre, mientras que en las reuniones diarias se comparará el avance real con la fecha límite de cada tarea. Si una tarea no ha iniciado 48 horas antes de su fecha planificada o permanece bloqueada durante más de un día, deberá reasignarse, dividirse

o retirarse del alcance. También se reservarán los dos últimos días exclusivamente para integración, pruebas, correcciones, documentación y preparación de la demostración, evitando programar desarrollo principal cerca del cierre. Estas medidas permitirán detectar desviaciones antes, reducir la acumulación de trabajo y producir un incremento más estable y verificable.